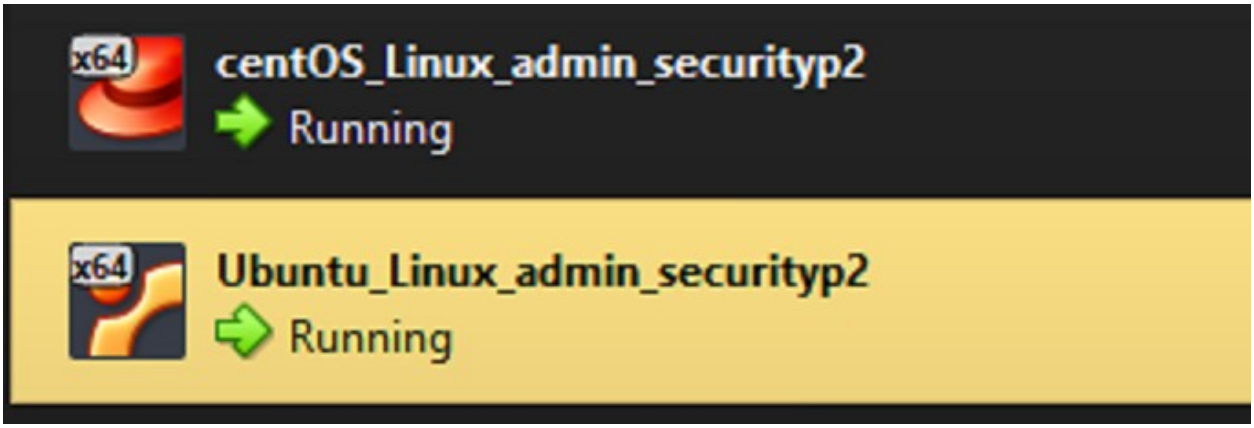
Joel Rodriguez
Fall 2026
CIS-245
Security Part 2

CentOS

1.1 Preparing the Virtual Machine for Troubleshooting

Before beginning the Server Troubleshooting Lab, I created a clone of my original Linux virtual machines using Oracle VirtualBox. I kept the original virtual machines powered off to preserve their existing configuration and protect my previous coursework. I used the cloned virtual machines for this assignment so that the troubleshooting script could make changes without affecting the original system.

I then transferred the provided `B0rk.zip` file to the cloned servers. This ZIP file contains the `break_system.sh` script that will introduce a problem for me to investigate. At this stage, I had not executed the script or begun troubleshooting.



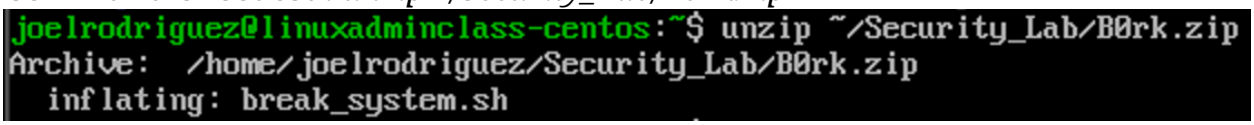
Source: https://www.virtualbox.org/manual/topics/working-with-vms.html#clone

1.2 Extracting the Troubleshooting Script

I extracted the professor-provided ZIP file on my CentOS server using the command `unzip ~/Security_Lab/B0rk.zip`. The `unzip` command extracts files from a ZIP archive, while `~` represents my home directory. The `/Security_Lab/B0rk.zip` portion identifies the location of the archive. The terminal output displayed `inflating: break_system.sh`, confirming that the script

was extracted successfully. I captured a screenshot of the command and its output as evidence.

Command executed: *unzip ~/Security_Lab/B0rk.zip*

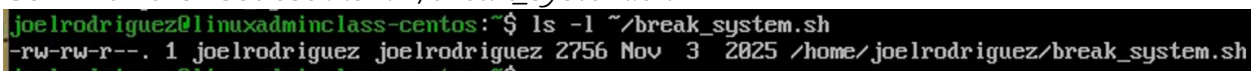


Source: https://linux.die.net/man/1/unzip

1.3 Verifying the Extracted Script

After extracting the ZIP archive, I verified that the troubleshooting script existed by running `ls -l ~/break_system.sh`. The `ls` command lists files, while the `-l` option displays detailed information, including file permissions, ownership, size, and modification date. The `~` symbol represents my home directory, and `/break_system.sh` identifies the file being checked. The output confirmed that the script was located in `/home/joelrodriguez/` and owned by my user account. Its permissions were `-rw-rw-r--`, indicating that the file was readable and writable by the owner and group, but did not have executable permission. This verified that the script was present before I attempted to run it.

Command executed: *ls -l ~/break_system.sh*

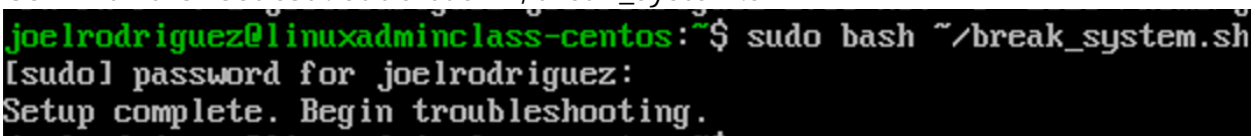


Source: https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html

1.4 Executing the Troubleshooting Script

After verifying that the script was available, I executed the professor-provided troubleshooting script on my CentOS clone using `sudo bash ~/break_system.sh`. The `sudo` command runs a command with elevated privileges, while `bash` executes the shell script. The `~` symbol represents my home directory, and `/break_system.sh` identifies the script being executed. After entering my administrator password, the terminal displayed "Setup complete. Begin troubleshooting." This confirmed that the script completed its setup process and that the server was ready for the troubleshooting portion of the assignment. I captured a screenshot of the command and output as evidence.

Command executed: sudo bash ~/break_system.sh

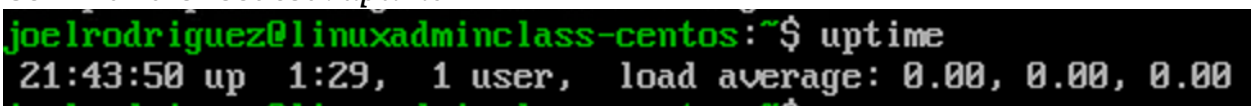


Sources: https://www.gnu.org/software/bash/manual/
https://www.sudo.ws/docs/

2.1 Checking System Uptime

I began my CentOS system health investigation by running the `uptime` command, which displays how long the server has been running, the number of logged-in users, and the system load averages. The output showed that my server had been running for 1 hour and 29 minutes with one active user. The load averages for the previous 1, 5, and 15 minutes were all 0.00, indicating very little system load during those periods. This provided an initial overview of the server's activity, although it did not establish whether other system components were functioning correctly.

Command executed: *uptime*



Source: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html

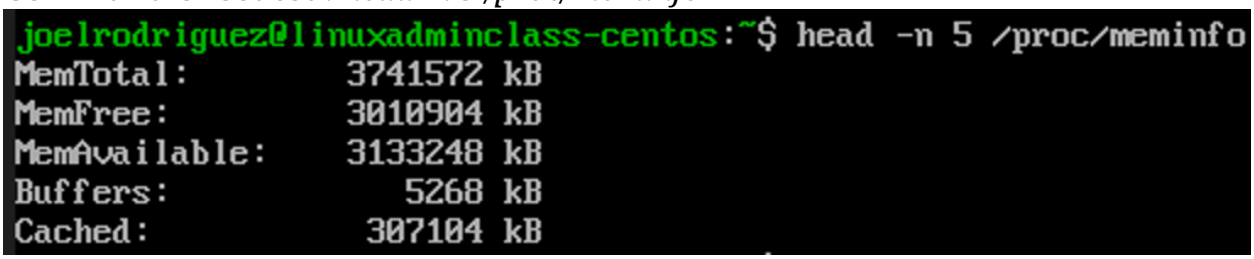
2.2 Checking System Memory

I continued examining my CentOS server's health by running `cat /proc/meminfo` to view detailed memory information. The `cat` command displays the contents of a file, while `/proc/meminfo` is a virtual file that provides information about system memory. Since the output was lengthy, I also ran `head -n 5 /proc/meminfo`. The `head` command displays the beginning of a file, and the `-n 5` option limits the output to five lines.

The results showed 3,741,572 kB of total memory, with 3,133,248 kB available. This meant approximately 83.7% of the server's memory was available at the time of testing. The earlier output also showed that all 3,358,716 kB of configured

swap space was free. Based on these observations, the server did not appear to be experiencing memory pressure during my checks. However, these results alone did not establish the cause of the server's problem.

Command executed: *head -n 5 /proc/meminfo*



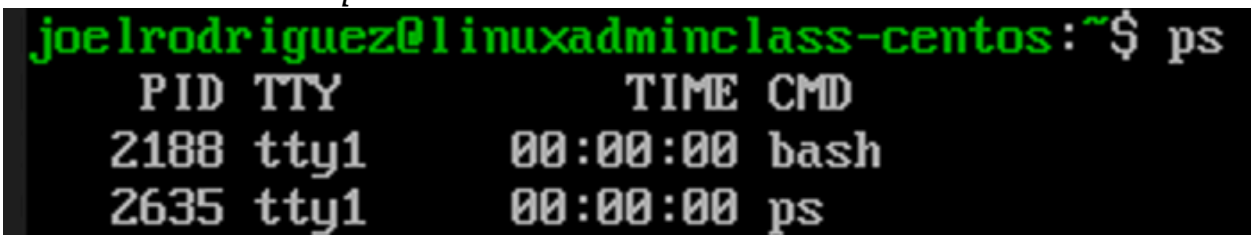
Sources: https://docs.kernel.org/filesystems/proc.html

2.3 Checking Running Processes

I continued reviewing my CentOS server's health by running the `ps` command, which displays information about active processes. The output contained four columns: PID, TTY, TIME, and CMD. PID identifies the process, TTY identifies its associated terminal, TIME shows the CPU time used, and CMD displays the command name.

The results showed two processes associated with my terminal: `bash` with PID 2188 and `ps` with PID 2635. Both displayed 00:00:00 of CPU time. Since I ran `ps` without additional options, the output was limited to processes associated with my terminal session. This confirmed that the command was working but did not provide a complete picture of all processes running on the server.

Command executed: *ps*



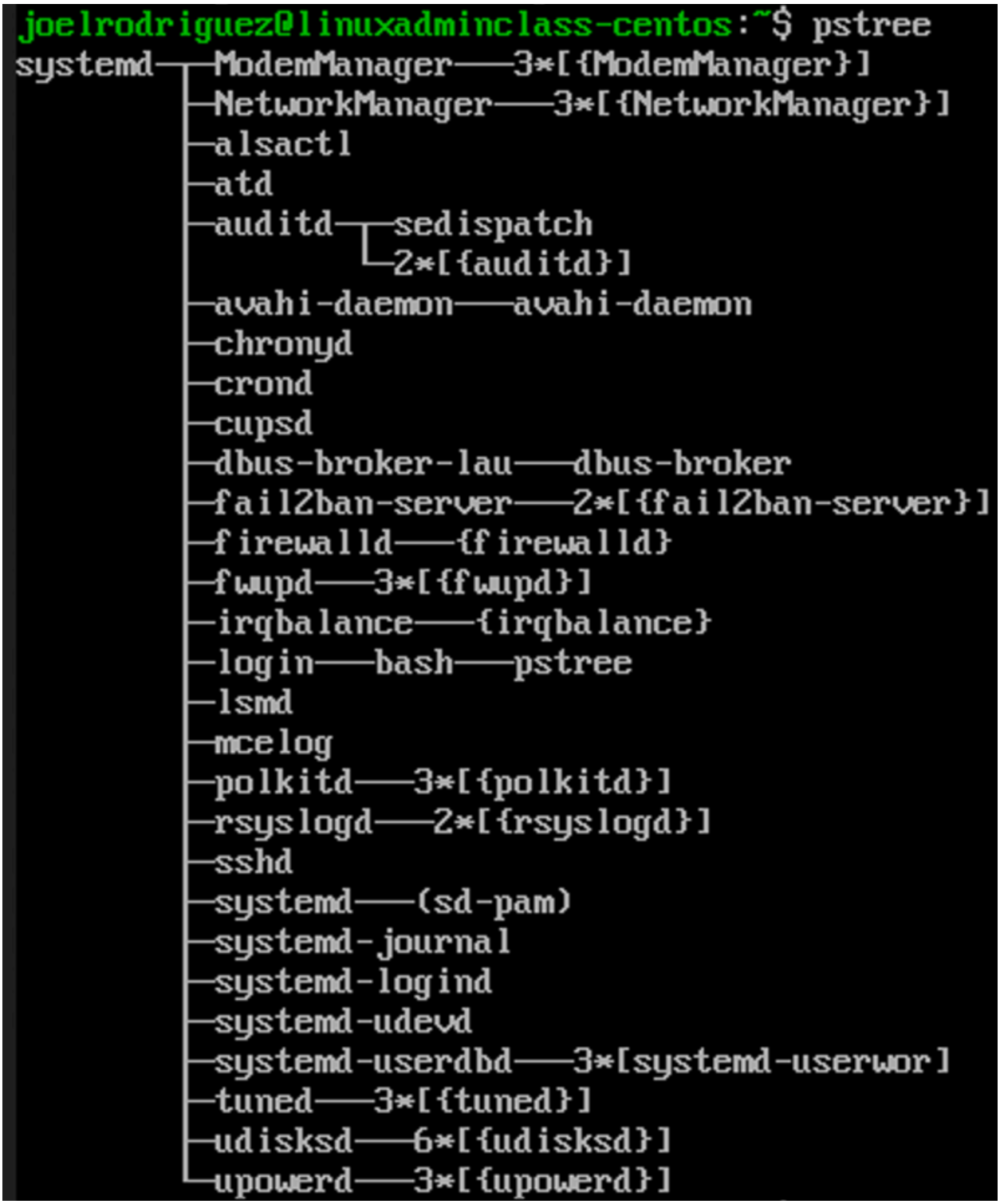
Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://man7.org/linux/man-pages/man1/ps.1.html

2.4 Examining the Process Tree

I continued monitoring my CentOS server by running the `pstree` command to display running processes in a hierarchical structure. Unlike the `ps` command, which previously showed only processes associated with my terminal, `pstree` displayed parent and child process relationships throughout the system.

The output showed `systemd` at the top of the process tree, with several processes underneath it, including `NetworkManager`, `sshd`, `firewalld`, `crond`, `rsyslogd`, and `auditd`. I also observed my terminal session represented by `login`, followed by `bash` and `pstree`. This helped me understand how the server organizes running processes. However, the process tree alone did not confirm that every service was functioning correctly or identify the cause of the server's problem.
Command executed: *pstree*

Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://man7.org/linux/man-pages/man1/pstree.1.html


joelrodriguez@linuxadminclass-centos:~$ pstree
systemd─┬─ModemManager───3*[{ModemManager}]
├─NetworkManager───3*[{NetworkManager}]
├─alsactl
├─atd
├─auditd─┬─sedispatch
│ └─2*[{auditd}]
├─avahi-daemon───avahi-daemon
├─chronyd
├─crond
├─cupsd
├─dbus-broker-lau───dbus-broker
├─fail2ban-server───2*[{fail2ban-server}]
├─firewalld───{firewalld}
├─fwupd───3*[{fwupd}]
├─irqbalance───{irqbalance}
├─login───bash───pstree
├─lsmd
├─mcelog
├─polkitd───3*[{polkitd}]
├─rsyslogd───2*[{rsyslogd}]
├─sshd
├─systemd───(sd-pam)
├─systemd-journal
├─systemd-logind
├─systemd-udevd
├─systemd-userdbd───3*[systemd-userwor]
├─tuned───3*[{tuned}]
├─udisksd───6*[{udisksd}]
└─upowerd───3*[{upowerd}]

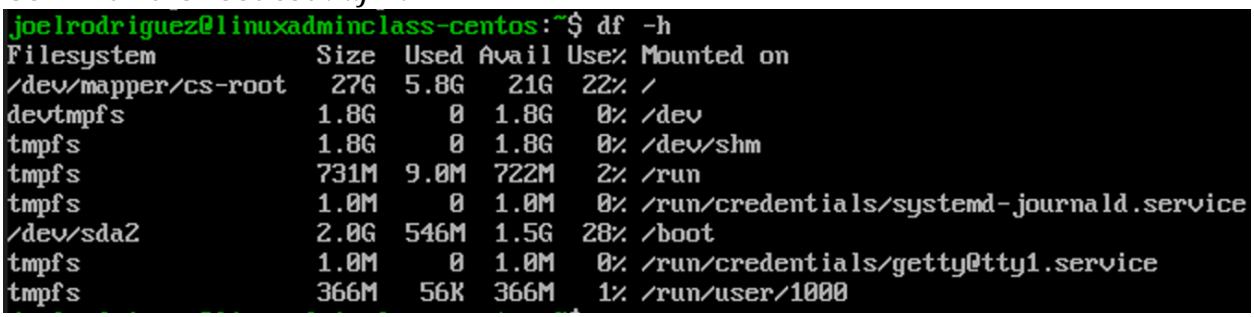
2.5 Checking Filesystem Disk Usage

I continued examining my CentOS server's health by running `df -h` to review filesystem disk usage. The `df` command reports disk space usage for mounted filesystems, while the `-h` option displays storage sizes in a human-readable format, such as megabytes and gigabytes.

The output showed that the root filesystem (`/dev/mapper/cs-root`) had a total capacity of 27 GB, with 5.8 GB used and approximately 21 GB available, representing 22% utilization. The `/boot` filesystem (`/dev/sda2`) had a total capacity of 2.0 GB, with 546 MB used and approximately 1.5 GB available, representing 28% utilization.

Based on these results, neither filesystem appeared to be running low on available disk space at the time of testing. However, this command alone could not confirm whether other filesystem-related problems existed.

Command executed: *df -h*



Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://www.gnu.org/software/coreutils/manual/html_node/df-invocation.html

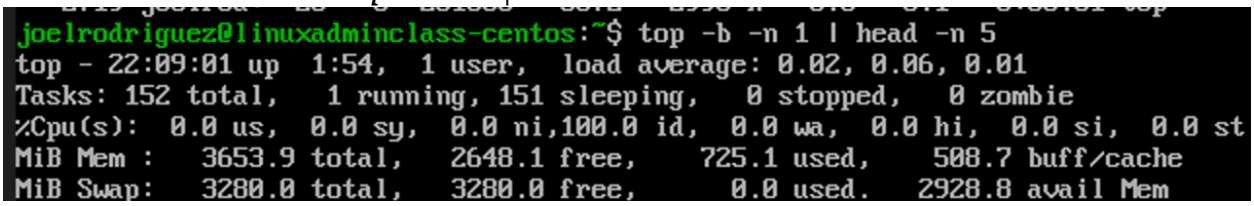
2.6 Reviewing Overall CPU Utilization

I continued monitoring my CentOS server by running `top -b -n 1 | head -n 5` to display a summary of CPU, memory, and process activity. The `top` command displays system resource utilization, while `-b` enables batch mode and `-n 1` limits the command to one snapshot. The pipe symbol (`|`) passes the output to `head`, and `-n 5` limits the displayed results to the first five lines.

The output showed 152 total tasks, with one running and 151 sleeping. CPU idle time was 100.0%, and the load averages were 0.02, 0.06, and 0.01. The server also had approximately 2,928.8 MiB of available memory and no swap usage.

Based on these results, I did not observe excessive CPU or memory utilization during this monitoring exercise. However, the results did not establish whether all server services and other components were functioning correctly.

Command executed: *top -b -n 1 | head -n 5*



Sources: https://web.archive.org/web/20220516044302/https://linuxconfig.org/how-to-check-and-monitor-cpu-utilization-on-linux
https://web.archive.org/web/20221012133324/https://www.linuxfoundation.org/blog/blog/classic-sysadmin-linux-101-5-commands-for-checking-memory-usage-in-linux
https://man7.org/linux/man-pages/man1/top.1.html
https://www.gnu.org/software/coreutils/manual/html_node/head-invocation.html

2.7 Reviewing Network Interfaces

I continued monitoring my CentOS server by running `ip -br addr` to review its network interfaces and assigned IP addresses. The `ip` command displays network configuration information, while the `-br` option presents the results in a brief format. The `addr` argument displays the IP addresses assigned to each interface.

The output showed two interfaces: `lo` and `enp0s3`. The `lo` interface had the loopback address `127.0.0.1/8` and displayed a state of UNKNOWN, which is not necessarily an error for a loopback interface. The `enp0s3` interface displayed a state of UP and had the IPv4 address `10.0.0.173/24`, along with IPv6 addresses.

Based on these results, I confirmed that my CentOS server had an enabled network interface with an assigned IP address. However, this information alone did not verify connectivity to other systems.

Command executed: *ip -br addr*



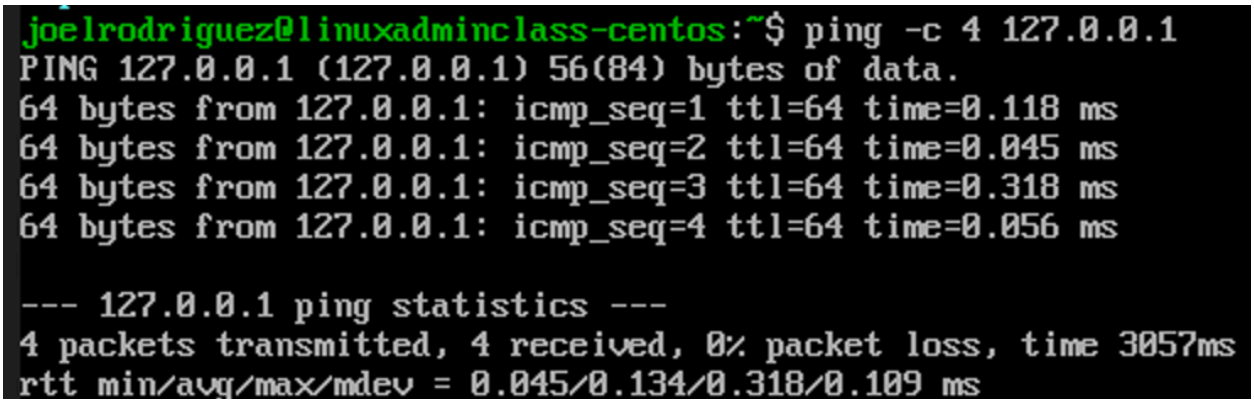
Sources: https://www.aholdengouveia.name/LinuxAdmin/networking.html
https://man7.org/linux/man-pages/man8/ip-address.8.html

2.8 Testing Local Network Connectivity

I continued reviewing my CentOS server's network configuration by running `ping -c 4 127.0.0.1` to test communication with the local loopback interface. The `ping` command sends ICMP echo requests to a specified destination, while the `-c 4` option limits the test to four requests. The IP address `127.0.0.1` represents the local loopback interface, allowing me to test communication within the server itself without contacting another machine.

The results showed that all four packets were transmitted and received successfully, with 0% packet loss and an average response time of 0.134 milliseconds. Based on these results, I confirmed that the server's loopback interface responded correctly during the test. However, this did not verify connectivity to external systems.

Command executed: *ping -c 4 127.0.0.1*



Sources: https://www.aholdengouveia.name/LinuxAdmin/networking.html https://man7.org/linux/man-pages/man8/ping.8.html

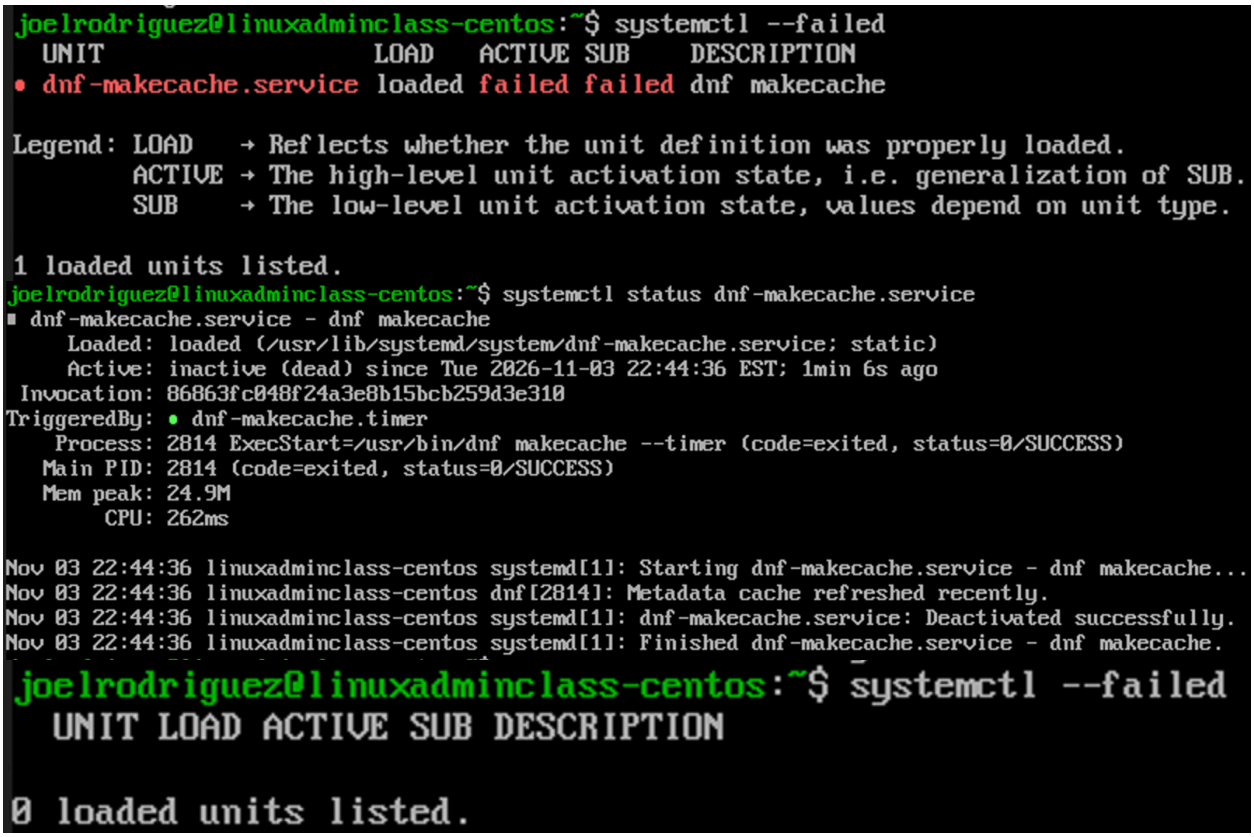
2.9 Investigating Failed System Services

I continued investigating my CentOS server by running systemctl --failed to identify systemd units reporting a failed state. The systemctl command manages and examines systemd services, while the --failed option displays units that have entered a failed state.

The initial output identified dnf-makecache.service as failed. To examine this further, I ran systemctl status dnf-makecache.service, which displayed the service's current state, execution information, and recent log messages. The output showed that the service had completed its latest execution with an exit status of 0/SUCCESS and was inactive afterward.

I then ran systemctl --failed again to verify whether the failure remained. This time, the output displayed 0 loaded units listed, confirming that no systemd units were currently reporting a failed state. Although the earlier service failure had cleared, I had not yet determined whether it was related to the problem introduced by the professor's script.

Commands executed: *systemctl –failed and systemctl status dnf-makecache.service*



Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html

2.10 Investigating and Restoring the Apache Web Server

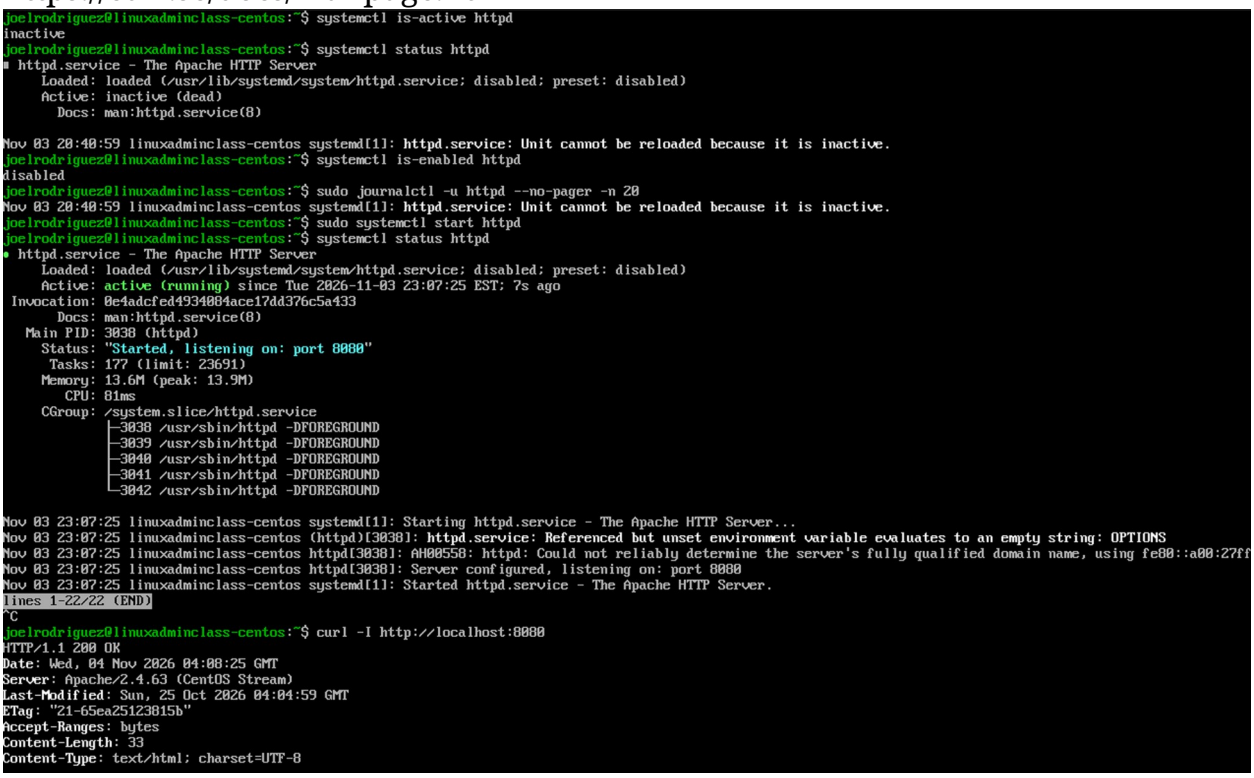
During my CentOS troubleshooting investigation, I examined the Apache web server after discovering that its configuration file differed from the installed package's original version. I first ran `*sudo apachectl configtest*` to check the Apache configuration for syntax errors. The `sudo` command provides elevated privileges, while `apachectl configtest` checks the configuration without starting the service. The output displayed a ServerName warning followed by `Syntax OK`, indicating that Apache's configuration passed its syntax check.

I then ran `*systemctl is-active httpd*` and `*systemctl status httpd*` to examine the service's operating state. Both checks confirmed that Apache was inactive. I also ran `*systemctl is-enabled httpd*`, which returned `disabled`, indicating that Apache was not configured to start automatically during boot. To investigate further, I used `*sudo journalctl -u httpd --no-pager -n 20*` to review the 20 most recent Apache journal entries. The output showed that a previous attempt to reload Apache had failed because the service was inactive.

After identifying that Apache was stopped, I ran `*sudo systemctl start httpd*` to start the service. I verified that its status changed to `active (running)`. Finally, I ran `curl -I http://localhost:8080` to test whether Apache was responding to HTTP requests on port 8080. The command returned `HTTP/1.1 200 OK`, confirming that the web server was responding successfully.

These results demonstrated that I restored Apache's local HTTP functionality. However, further verification was still necessary to determine whether this resolved the problem introduced by the troubleshooting script.

Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://httpd.apache.org/docs/2.4/
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html
https://www.freedesktop.org/software/systemd/man/latest/journalctl.html
https://curl.se/docs/manpage.html



2.11 Investigation of the Four Possible System Failures

After restoring the Apache service, I continued investigating my CentOS server because I had not yet identified which problem the professor's `break_system.sh` script introduced. Although the server was running and Apache responded successfully, I wanted to determine whether any other system configurations had been affected.

2.11.1 Reviewing the Troubleshooting Script

Commands executed:
cat ~/break_system.sh
cat ~/break_system.sh | head -n 20
cat ~/break_system.sh | head -n 25
cat ~/break_system.sh | head -n 30
cat ~/break_system.sh | head -n 40
cat ~/break_system.sh | head -n 45

The `cat` command displayed the contents of the script. The pipe (`|`) passed the output to `head`, while `-n` specified the number of lines displayed.

I discovered that the script uses `SCENARIO=$((RANDOM % 4 + 1))` to randomly select one of four scenarios: networking problems, boot configuration problems, file permission problems, storage and log rotation problems.

This helped me focus my investigation on the actual changes the script was designed to make.

2.11.2 Investigating Network Configuration — Scenario 1

I executed:
cat /etc/resolv.conf
ip route

The first command displayed the DNS configuration, while the second displayed the IPv4 routing table.

The DNS configuration contained addresses including `75.75.75.75` and `75.75.76.76`, rather than the invalid `999.999.999.999` address used by the script.

The routing table showed a default gateway of `10.0.0.1` through interface `enp0s3`, with the server using IP address `10.0.0.173`.

Conclusion: I did not find the specific DNS or routing changes associated with Scenario 1 in the current configuration. However, these settings could have been regenerated after the server rebooted.

2.11.3 Investigating Boot Configuration — Scenario 2

I executed:
cat /etc/fstab
sudo grep -n 'root=UUID=broken' /boot/grub/grub.cfg
sudo ls -l /boot/grub2/grub.cfg
sudo grep -n 'root=UUID=broken' /boot/grub2/grub.cfg

The `cat /etc/fstab` command displayed the configured filesystems. The file contained entries for the root filesystem, `/boot`, and swap. I did not find the invalid `/dev/sda99` entry that the script would have added.

I then used `grep` to search for the invalid boot parameter `root=UUID=broken`. The `-n` option would display the line number if a match was found. My first search returned `No such file or directory` because `/boot/grub/grub.cfg` did not exist.

I checked `/boot/grub2/grub.cfg` using `ls -l`, which confirmed that the GRUB configuration file existed at that location.

Finally, I searched the actual GRUB configuration for `root=UUID=broken`. The command returned no matching lines.

Conclusion: I found no evidence of the specific filesystem or GRUB modifications shown in Scenario 2.

2.11.4 Investigating File Permissions — Scenario 3

I executed:
ls -l /bin/ls /bin/cat
ls -l /etc/passwd /etc/shadow /etc/sudoers
ls -ld /etc/sshd_config
ls -l /etc/ssh

The `ls` command displayed file information, while `-l` provided detailed permissions and ownership. The `-d` option displayed information about the specified path itself.

Both `/bin/ls` and `/bin/cat` had executable permissions of `755` and were owned by root.

The `/etc/passwd` file had permissions of `644`, `/etc/shadow` had permissions of `000`, and `/etc/sudoers` had permissions of `440`. All three were owned by root.

My attempt to access `/etc/sshd_config` failed because that path did not exist. I then listed `/etc/ssh` and found the actual SSH server configuration file at `/etc/ssh/sshd_config`, with permissions of `600` and root ownership.

Conclusion: The inspected files did not show the specific permission and ownership changes the script attempts under Scenario 3. I did not inspect every possible SSH-related change.

2.11.5 Investigating Storage and Log Rotation — Scenario 4

I executed:
ls -lh /var/log/bigfile /tmp/hugefile
cat /etc/logrotate.conf
ls -l /forcefsckkkk
ls -l /forcefsck

The first command checked for the two large files that the script attempts to create. The `-h` option displays file sizes in a human-readable format.

Both files returned `No such file or directory`.

I then displayed `/etc/logrotate.conf` to inspect the log rotation configuration. The file contained normal configuration directives, including weekly rotation, retention of four rotated logs, and inclusion of `/etc/logrotate.d`.

I initially mistyped the filesystem-check marker as `/forcefsckkkk`. After correcting the command to `/forcefsck`, the system reported that the file did not exist.

Conclusion: I did not find the specific large files, log rotation corruption, or filesystem-check marker associated with Scenario 4.

2.11.6 Reviewing Command History

I executed: *history | tail -n 30*

The `history` command displayed previously executed Bash commands. The pipe passed the output to `tail`, and `-n 30` limited the display to the final 30 entries.

The screenshot showed history entries 623 through 652, covering the post-reboot Apache checks and my investigation of the four possible scenarios.

The history also included `history -d 636`, an attempt to delete a history entry. I retained the screenshot as part of the troubleshooting record.

2.11.7 Final Investigation Summary

After reviewing the four scenarios, I did not find conclusive evidence identifying which scenario had originally been selected on CentOS.

The network configuration appeared functional, the inspected filesystem and boot configuration files did not contain the scripted invalid entries, the critical file permissions appeared normal, and the storage-related files were absent.

Because the server had been rebooted, some temporary changes could have disappeared or been regenerated. Additionally, some commands in the script were conditional on files or directories existing.

The Apache service had been restored and configured to start automatically, and an HTTP request returned `200 OK`. However, I could not conclusively establish that the Apache problem was caused by the professor's script.

I preserved the troubleshooting screenshots and command history to document both the successful checks and the remaining uncertainty.

Sources: System Breaker Script — “break_system.sh”
https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://www.freedesktop.org/software/systemd/man/latest/

```
joelrodriguez@linuxadminclass-centos:~$ history | tail -n 30
  623  systemctl is-system-running
  624  systemctl --failed
  625  systemctl is-active httpd
  626  sudo systemctl enable --now httpd
  627  systemctl is-active httpd
  628  curl -I http://localhost:8080
  629  history | tail -n 20
  630  cat ~/break_system.sh
  631  cat ~/break_system.sh | head -n 20
  632  cat ~/break_system.sh | head -n 25
  633  cat ~/break_system.sh | head -n 30
  634  cat ~/break_system.sh | head -n 40
  635  cat ~/break_system.sh | head -n 45
  636  cat /etc/resolv.conf
  637  ip route
  638  cat /etc/fstab
  639  ls -l /bin/ls /bin/cat
  640  ls -l /etc/passwd /etc/shadow /etc/sudoers
  641  ls -lh /var/log/bigfile /tmp/hugefile
  642  cat /etc/logrotate.conf
  643  ls -l /forcefsckkkkk
  644  ls -l /forcefsck
  645  ls -ld /etc/sshd_config
  646  ls -l /etc/ssh
  647  sudo grep -n 'root=UUID=broken' /boot/grub/grub.cfg
  648  sudo ls -l /boot/grub2/grub.cfg
  649  sudo grep -n 'root=UUID=broken' /boot/grub2/grub.cfg
  650  history | tail -n 30
```

3. Automated System Diagnostics

For this portion of the ACME-6390 Report Server Troubleshooting Lab, I created a Bash script that automatically collects system diagnostic information. The purpose of this script was to make troubleshooting easier by gathering information about the operating system, CPU, memory, storage, networking, services, and system logs into a single report.

I developed the script on CentOS Stream 10 and then transferred it to Ubuntu to test whether it would work on both Linux distributions without modification. The script saves its results to a timestamped text file, allowing me to preserve diagnostic information for future troubleshooting.

CentOS Stream 10 — Script Creation and Testing

3.1 Creating the Bash Script

I created a Bash script named system_diag.sh in the following directory on my CentOS server:

/home/joelrodriguez/Security_Lab/scripts/

I used the GNU nano text editor to write the script.

The beginning of the script contained the following configuration:

```
#!/bin/bash

# Compatible with CentOS and Ubuntu
# Purpose: Collect health information for troubleshooting

REPORT_DIR="$HOME/Security_Lab/reports"
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_FILE="$REPORT_DIR/system_diagnostics_$TIMESTAMP.txt"

mkdir -p "$REPORT_DIR"
```

The #!/bin/bash line identifies Bash as the interpreter. The REPORT_DIR variable defines the directory where reports are saved.

The TIMESTAMP variable generates the date and time using the date command. This information is included in the report filename to distinguish reports generated at different times.

The REPORT_FILE variable stores the full path of the output file.

Finally, mkdir -p creates the reports directory if it does not already exist.

3.2 Section 1 — Basic System Information

The first section collects information about the Linux operating system.
The commands used were:
date
hostname
cat /etc/os-release
uname -r
uptime
I used date to record when the report was generated and hostname to identify the server.
The cat /etc/os-release command displays operating system information, while uname -r identifies the Linux kernel version.
The uptime command displays how long the system has been running and its current load averages.
This information provides a starting point for troubleshooting and helps identify the system being examined.

3.3 Section 2 — CPU and Memory Information

The second section collects CPU and memory information using:
lscpu
free -h
uptime
The lscpu command displays processor details, including CPU architecture and processor availability.
The free -h command displays memory and swap usage in a human-readable format.
The uptime command displays system load averages.
I included these commands to help identify performance problems related to CPU utilization, memory availability, or overall system load.

3.4 Section 3 — Disk Space Information

The third section checks filesystem and storage information using:
df -h
lsblk
The df -h command displays used and available filesystem space.
The lsblk command lists block devices and partitions.
These commands are useful for identifying disk space shortages and examining the server's storage configuration.

3.5 Section 4 — Network Diagnostics
The fourth section performs several network checks using:
ip addr
ip route
cat /etc/resolv.conf
ping -c 4 -W 2 1.1.1.1
getent hosts google.com
The ip addr command displays network interfaces and their assigned addresses.
The ip route command displays the routing table, including the default gateway when one is available.
The cat /etc/resolv.conf command displays the system's DNS resolver configuration.
The ping command sends four ICMP echo requests to 1.1.1.1 to test external network connectivity. The -W 2 option sets a two-second response timeout.
Finally, getent hosts google.com checks whether the server can resolve a domain name.
I included these commands because network problems can prevent servers from communicating with other systems or accessing required services.

3.6 Section 5 — Systemd Services
The fifth section checks systemd services using:
systemctl is-system-running
systemctl --failed --no-pager
systemctl list-units --type=service --state=running --no-pager
The systemctl is-system-running command reports the overall system state.

The systemctl --failed --no-pager command lists failed systemd units.
The final command lists running services without opening an interactive pager.
These checks help identify services that may have stopped unexpectedly or failed during startup.

3.7 Section 6 — System Logs and Errors
The sixth section collects journal messages using:
journalctl -p err -n 20 --no-pager
journalctl -b -p warning -n 20 --no-pager
The first command retrieves up to 20 recent error-priority journal entries.
The second command retrieves up to 20 warning-or-higher-priority entries from the current boot.
I included these commands because system logs often contain useful information about service failures, networking problems, and hardware-related warnings.

3.8 Section 7 — Final Report Summary
The final section records the completion time and hostname, followed by a message reminding the user to review the report for potential issues.
Each diagnostic section appends its output to the report file using:
} >> "$REPORT_FILE" 2>&1
The >> operator appends output to the existing report file, while 2>&1 redirects standard error to the same destination as standard output.
This allows the report to include both normal command output and error messages.
After completing the report, the script displays a success message and the location of the saved file.

3.9 Checking the Script Syntax
After saving the script, I checked its Bash syntax using:
bash -n ~/Security_Lab/scripts/system_diag.sh
The command returned to the terminal prompt without displaying an error.
This indicated that Bash did not detect a syntax error during the check.
I also verified the file using:
ls -l ~/Security_Lab/scripts/system_diag.sh
The terminal displayed the script's file information, confirming that it was saved in the intended directory.

3.10 Executing the Script on CentOS
I executed the script using:

bash ~/Security_Lab/scripts/system_diag.sh
The terminal displayed:
Diagnostics completed successfully.
Report saved to:
/home/joelrodriguez/Security_Lab/reports/system_diagnostics_2026-11-05_15-40-25.txt
This confirmed that the script reached its completion message and reported the location of the generated diagnostic file.
The report was saved in the Security_Lab/reports directory with a timestamp in its filename.

3.11 Verifying the CentOS Report
After running the script, I inspected the generated report and checked whether it contained all seven diagnostic sections.
I used grep -n to search for the section headings and display their line numbers.
The report contained the following sections:

Section	Starting line
System Information	1
CPU and Memory Information	26
Disk Space Information	78
Network Diagnostics	99
Systemd Services	124
System Logs and Errors	174
Diagnostics Report Complete	212

This confirmed that the script generated the expected report structure.
3.12 Reviewing the CentOS Diagnostic Results
I reviewed the system logs collected by the script.
The report contained several warnings and errors, including NetworkManager connection activation failures and virtual graphics-related messages.
The NetworkManager messages indicated that the system had experienced difficulty activating a network connection.
The graphics warnings suggested that the virtual machine's graphics configuration might not be fully compatible with the virtualized environment.
These entries demonstrated that the diagnostic script could collect useful troubleshooting evidence.

However, the presence of historical warnings does not necessarily mean that every reported problem was still active when the script ran.

3.13 CentOS Conclusion

I successfully created and executed the automated diagnostic script on CentOS Stream 10.
The script generated a timestamped report containing seven categories of system information.
The report included system configuration details, resource information, networking data, service status, and journal messages.
This confirmed that the script could collect useful diagnostic information on CentOS and provided the initial test before transferring it to Ubuntu.

Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://www.aholdengouveia.name/LinuxAdmin/networking.html
https://web.archive.org/web/20221012133324/
https://www.linuxfoundation.org/blog/blog/classic-sysadmin-linux-101-5-commands-for-checking-memory-usage-in-linux
https://web.archive.org/web/20220516044302/https://linuxconfig.org/how-to-check-and-monitor-cpu-utilization-on-linux
https://web.archive.org/web/20220607232203/https://www.networkworld.com/article/3330497/linux-commands-for-measuring-disk-activity.html
https://www.gnu.org/software/bash/manual/bash.htmlhttps://web.archive.org/web/20220510004952/https://www.networkworld.com/article/3428361/how-to-manage-logs-in-linux.html
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html
https://man7.org/linux/man-pages/man1/journalctl.1.html
https://www.gnu.org/software/bash/manual/html_node/Redirections.html
https://www.gnu.org/software/grep/manual/grep.html

```
#!/bin/bash

# Compatible with CentOS and Ubuntu
# Purpose: Collect health information for troubleshooting

REPORT_DIR="$HOME/Security_Lab/reports"
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_FILE="$REPORT_DIR/system_diagnostics_$TIMESTAMP.txt"

mkdir -p "$REPORT_DIR"

# Section 1: Basic System Informaton

{
        echo "======= SYSTEM INFORMATION ======="
        echo "Report generated: $(date)"
        echo "Hostname: $(hostname)"
        echo "Operating System: "
        cat /etc/os-release
        echo "Kernel version: $(uname -r)"
        echo "System uptime: "
        uptime
} >> "$REPORT_FILE" 2>&1

# Section 2: CPU and Memory Information

{
        echo " "
        echo "======= CPU AND MEMORY INFORMATION ======="
        echo "CPU details: "
        lscpu
        echo "Memory usage: "
        free -h
        echo "System load: "
        uptime
} >> "$REPORT_FILE" 2>&1
```

```
# Section 3: Disk Space Information

{
        echo " "
        echo "======= DISK SPACE INFORMATION ======="
        echo "Filesystem disk usage: "
        df -h
        echo "Disk and partition information: "
        lsblk
} >> "$REPORT_FILE" 2>&1

# Section 4: Network Diagnostics

{
        echo " "
        echo "======= NETWORK DIAGNOSTICS ======="
        ip addr
        echo "Routing table: "
        ip route
        echo "DNS configuration: "
        cat /etc/resolv.conf
        echo "External connectivity test: "
        ping -c 4 -W 2 1.1.1.1
        echo "DNS resolution test: "
        getent hosts google.com
} >> "$REPORT_FILE" 2>&1
```

```
# Section 5: Systemd Services

{
        echo " "
        echo "======= SYSTEMD SERVICES ======="
        echo "Overall system status: "
        systemctl is-system-running
        echo "Failed services: "
        systemctl --failed --no-pager
        echo "Running services: "
        systemctl list-units --type=service --state=running --no-pager
} >> "$REPORT_FILE" 2>&1

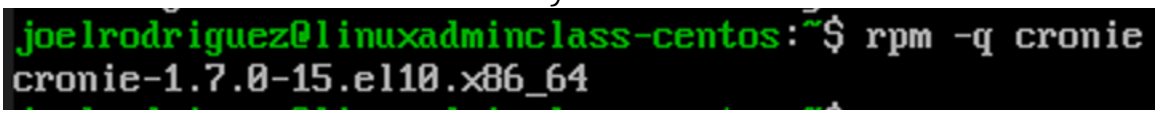
# Section 6: System Logs and Errors

{
        echo " "
        echo "======= SYSTEM LOGS AND ERRORS ======="
        echo "Recent system errors: "
        journalctl -p err -n 20 --no-pager
        echo "Recent boot warnings and errors: "
        journalctl -b -p warning -n 20 --no-pager
} >> "$REPORT_FILE" 2>&1

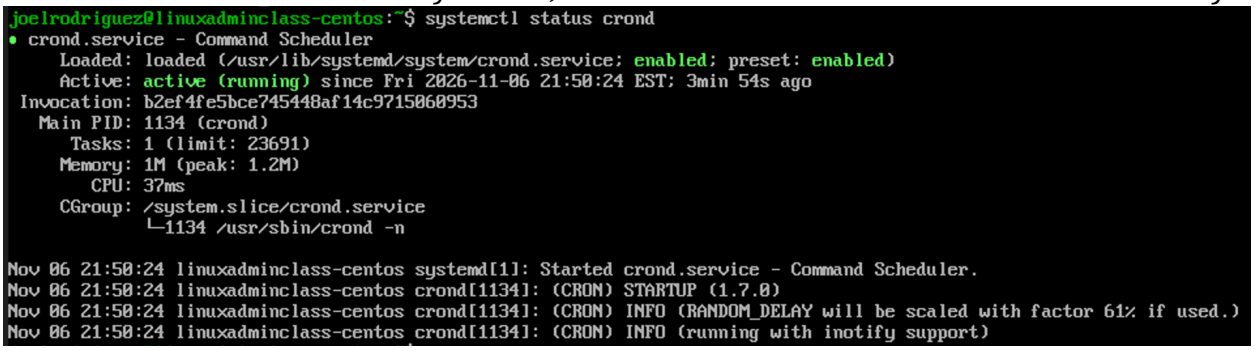
# Section 7: Final Report Summary
{
        echo " "
        echo "======= DIAGNOSTICS REPORT COMPLETE ======="
        echo "Report completed: $(date)"
        echo "Hostname: $(hostname)"
        echo "Review the sections above for potential issues."
} >> "$REPORT_FILE" 2>&1

echo "Diagnostics completed successfully."
echo "Report saved to: $REPORT_FILE"
```

4. Automated Diagnostic Scheduling with Cron
CentOS Stream 10 — Cron Configuration and Testing
4.1 Verifying Cron Installation
After successfully creating and testing my Bash diagnostic script, I configured CentOS Stream 10 to execute the script automatically using cron. Cron is a Linux scheduling utility that allows administrators to execute commands and scripts at specified times without manual intervention.
Before configuring the schedule, I checked whether the Cronie package was installed using:
rpm -q cronie
The terminal returned:
cronie-1.7.0-15.el10.x86_64
This confirmed that Cronie was already installed on my CentOS server, so no additional installation was necessary.



4.2 Checking the Cron Service
Next, I checked whether the cron scheduler was running using:
systemctl status crond
The output showed:
Loaded: loaded (.../crond.service; enabled; preset: enabled)
Active: active (running)
This confirmed that the crond service was running and enabled to start automatically during system startup.
Since the service was already active, I did not need to start or enable it manually.



4.3 Checking Existing Scheduled Jobs

Before creating a new scheduled task, I checked whether my user account already had any cron jobs configured.

I executed:

crontab -l

The terminal returned:

no crontab for joelrodriguez

This indicated that my user account did not have an existing personal crontab. I performed this check to avoid accidentally replacing or interfering with previously scheduled tasks.



4.4 Creating a Temporary Cron Schedule

To configure the automated diagnostic script, I opened my personal crontab using:

crontab -e

The command opened the Vim text editor with an empty crontab.

I pressed i to enter Insert mode and added the following entry:

** * * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh*

The five asterisks instruct cron to execute the command every minute.

I used the full path to Bash and the diagnostic script to ensure that cron could locate the required files without depending on my interactive terminal's working directory.

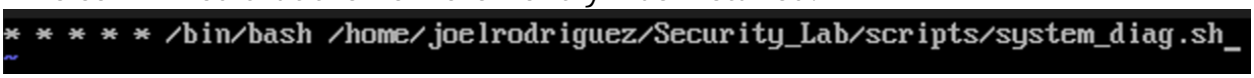
I selected a one-minute schedule temporarily so I could verify that the automation worked without waiting several hours.

After entering the command, I pressed Esc, typed :wq, and pressed Enter to save and exit Vim.

The terminal displayed:

crontab: installing new crontab

This confirmed that the new cron entry was installed.



```
"/tmp/crontab.eZmJ84" 1L, 76B written
crontab: installing new crontab
joelrodriguez@linuxadminclass-centos:~$
```

4.5 Verifying the Installed Cron Entry

After saving the schedule, I verified that the entry was installed correctly using:
crontab -l
The terminal displayed:
** * * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh*
This confirmed that the scheduled command was saved correctly and configured to execute every minute.

```
joelrodriguez@linuxadminclass-centos:~$ crontab -l
* * * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh
```

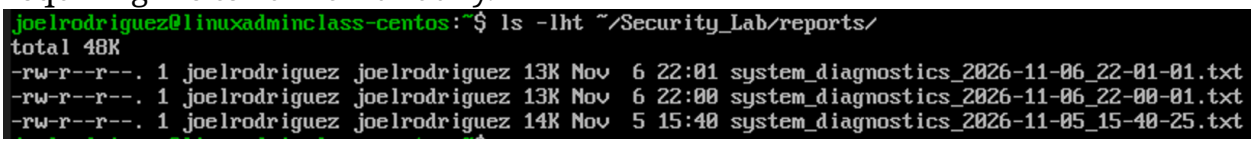
4.6 Testing Automatic Report Generation

After configuring cron, I checked whether the diagnostic script was generating reports automatically.
I executed:
ls -lht ~/Security_Lab/reports/
The terminal displayed three diagnostic reports:

Report filename	**File size**
system_diagnostics_2026-11-06_22-01-01.txt	13 KB
system_diagnostics_2026-11-06_22-00-01.txt	13 KB
system_diagnostics_2026-11-05_15-40-25.txt	14 KB

The two newer reports were generated at 10:00 PM and 10:01 PM, exactly one minute apart.
The older report was from my original manual test of the script.

The timestamps of the two new reports matched the temporary cron schedule, providing evidence that cron was executing the script automatically.
This confirmed that the diagnostic script could generate reports without requiring me to run it manually.



4.7 Changing the Schedule to Run Daily

After confirming that the temporary schedule worked, I changed the cron job to run once per day.
I reopened the crontab using:
crontab -e
I replaced the temporary one-minute schedule with:
*0 2 * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh*
The five scheduling fields represent the minute, hour, day of the month, month, and day of the week.
This configuration schedules the diagnostic script to execute daily at 2:00 AM according to the server's local time.
I selected a daily schedule because it allows regular system monitoring without generating unnecessary reports every minute.
After updating the entry, I saved and exited Vim.
The terminal confirmed that the updated crontab was installed.



4.8 Verifying the Final Cron Schedule
Finally, I verified the updated schedule using:
crontab -l
The terminal returned:
*0 2 * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh*
This confirmed that the temporary testing schedule had been replaced with the daily schedule.
The diagnostic script is now configured to run automatically at 2:00 AM each day, provided the CentOS server and cron service are running at that time.
The temporary one-minute schedule was tested successfully. The final daily schedule was verified as installed, but I did not wait until 2:00 AM to observe its first scheduled execution.

```
joelrodriguez@linuxadminclass-centos:~$ crontab -l
0 2 * * * /bin/bash /home/joelrodriguez/Security_Lab/scripts/system_diag.sh
```

4.9 CentOS Cron Configuration Conclusion
I successfully configured and tested cron scheduling on CentOS Stream 10.
I first verified that Cronie was installed and that the crond service was active and enabled.
I then created a temporary cron job to execute my diagnostic script every minute. By checking the reports directory, I confirmed that two diagnostic reports were generated exactly one minute apart.
After verifying that the automation worked, I changed the schedule to execute the script once daily at 2:00 AM.
This configuration allows the CentOS server to collect diagnostic information automatically and preserve timestamped reports for future troubleshooting.

Sources: https://man7.org/linux/man-pages/man5/crontab.5.html
https://man7.org/linux/man-pages/man1/crontab.1.html
https://man7.org/linux/man-pages/man1/crontab.1.html
https://www.process.st/templates/server-security-checklist/
https://www.aholdengouveia.name/LinuxAdmin/networking.html

CentOS Stream 10 — Emergency-Mode Recovery Guide

Executive Summary

During this lab, I successfully accessed emergency mode on my CentOS Stream 10 VirtualBox clone using the GRUB bootloader. I verified that the emergency target was active, inspected the root filesystem, and restarted the server successfully.

This procedure demonstrates how a Linux administrator can access a minimal recovery environment when normal startup or system services fail.

Emergency-Mode Recovery Procedure

1. Verify the normal boot configuration.
I checked the default systemd target and available GRUB boot entries:
systemctl get-default
sudo grubby --info=ALL
The default target was multi-user.target. GRUB displayed three standard kernel entries and one rescue entry.

2. Access emergency mode through GRUB.
I restarted the CentOS clone using sudo reboot and accessed the GRUB menu during startup. I selected the latest kernel (6.12.0-278.el10.x86_64) and pressed e to edit its boot parameters.
At the end of the linux command, I temporarily added:
systemd.unit=emergency.target
I pressed Ctrl+X to boot and accessed the root maintenance shell.

3. Verify the emergency environment.
At the root prompt, I executed:
systemctl is-active emergency.target
The result was active, confirming that CentOS was running in emergency mode.
I then inspected the root filesystem:
findmnt -no TARGET,OPTIONS /
The output began with / ro,relatime, confirming that the root filesystem was mounted read-only. This allows inspection while helping prevent unintended filesystem modifications. If configuration repairs were necessary, I would first evaluate whether remounting the filesystem as read-write was appropriate.

4. Return to normal operation.
After completing the verification, I restarted CentOS:
systemctl reboot
The server successfully returned to its normal login prompt. The temporary GRUB parameter did not permanently change the default boot configuration.

Figure 1:

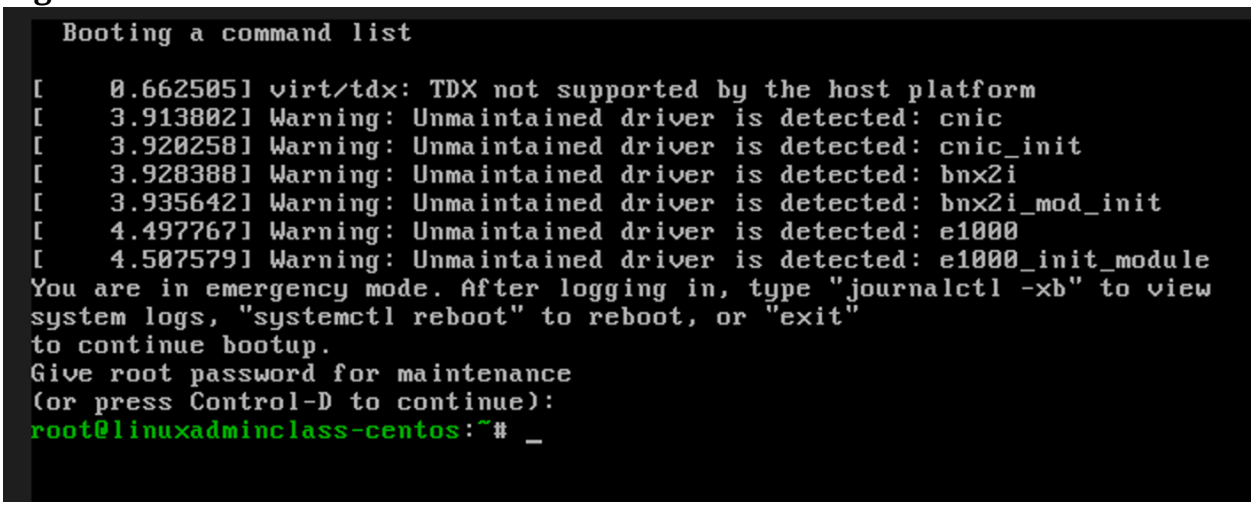


Figure 2:

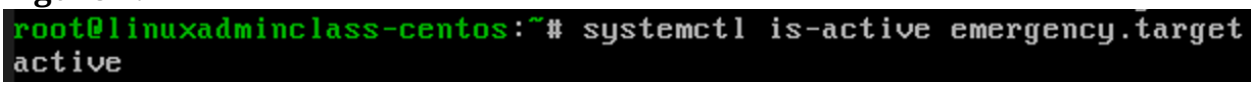
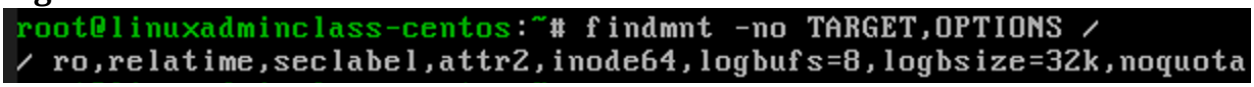


Figure 3:



Sources: https://docs.redhat.com/en/documentation/red_hat_enterprise_linux/10/html/using_systemd_unit_files_to_customize_and_optimize_your_system/index
https://www.freedesktop.org/software/systemd/man/latest/systemd.special.html
https://man7.org/linux/man-pages/man8/findmnt.8.html

UBUNTU

1.2 Extracting the Troubleshooting Script

I extracted the professor-provided ZIP archive on my Ubuntu server by running `unzip ~/Security_Lab/B0rk.zip`. The `unzip` command extracts files from a ZIP archive, while the `~` symbol represents my home directory. The `/Security_Lab/B0rk.zip` portion specifies the location of the archive. The terminal displayed `inflating: break_system.sh`, confirming that the script was successfully extracted. I captured a screenshot of the command and its output as evidence that the file was available on my Ubuntu server before beginning the troubleshooting portion of the lab.

Command executed: *unzip ~/Security_Lab/B0rk.zip*

```
jrodriguez@linuxadminclass-ubuntu:~$ unzip ~/Security_Lab/B0rk.zip
Archive:  /home/jrodriguez/Security_Lab/B0rk.zip
  inflating: break_system.sh
```

Source: https://linux.die.net/man/1/unzip

1.3 Verifying the Extracted Script

After extracting the ZIP archive, I verified that the troubleshooting script was present on my Ubuntu server by running `ls -l ~/break_system.sh`. The `ls` command lists files, while the `-l` option displays detailed information about a file, including its permissions, ownership, size, and modification date. The `~` symbol represents my home directory, and `/break_system.sh` specifies the file I wanted to examine. The output confirmed that the script was located at `/home/jrodriguez/break_system.sh`, was owned by my user account, and had permissions of `-rw-rw-r--`. These permissions indicate that the owner and group can read and write the file, while other users can only read it. The script did not have executable permission enabled. This confirmed that the file was available before beginning the troubleshooting portion of the lab.

Command executed: *ls -l ~/break_system.sh*

```
jrodriguez@linuxadminclass-ubuntu:~$ ls -l ~/break_system.sh
-rw-rw-r-- 1 jrodriguez jrodriguez 2756 Nov  3  2025 /home/jrodriguez/break_system.sh
```

Source: https://www.gnu.org/software/coreutils/manual/html_node/File-permissions.html

1.4 Executing the Troubleshooting Script

After confirming that the troubleshooting script was present on my Ubuntu server, I executed it using sudo bash ~/break_system.sh. The sudo command allows a command to run with elevated privileges, while bash runs the shell script. The ~ symbol represents my home directory, and /break_system.sh identifies the script being executed. After I entered my password, the terminal displayed "Setup complete. Begin troubleshooting." This confirmed that the professor's script had completed its setup process on my Ubuntu clone. I captured a screenshot of the command and its output before beginning my investigation.

Command executed: *sudo bash ~/break_system.sh*

```
jrodriguez@linuxadminclass-ubuntu:~$ sudo bash ~/break_system.sh
[sudo: authenticate] Password:
Setup complete. Begin troubleshooting.
```

Source: https://www.gnu.org/software/bash/manual/
https://www.sudo.ws/docs/

Section 2 — Ubuntu Server Troubleshooting and Repair

2.1 Executing the System Breaker Script

I began my investigation by running the professor-provided `break_system.sh` script on my Ubuntu virtual machine. The purpose of the script was to introduce a randomly selected system problem that I would need to identify and repair.

My Bash history recorded the following commands: sudo bash ~/break_system.s
ls
sudo bash ~/break_system.sh

The first command contained an incomplete filename. I then used `ls` to list the files in my current directory and executed the script again using its correct filename. The `sudo` command runs a command with elevated privileges, while `bash` executes the specified shell script. The successful execution of the script was followed by my investigation of the system's health and configuration.

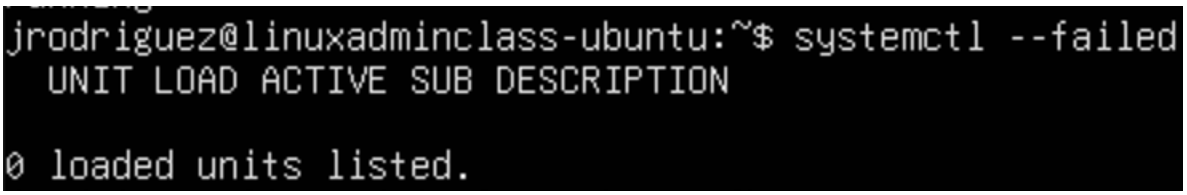
2.2 Checking Ubuntu's Overall System Health
I began by checking whether Ubuntu was running normally and whether any systemd services had failed.

systemctl is-system-running
systemctl --failed

The first command returned: running
The second command returned: 0 loaded units listed.

The `systemctl is-system-running` command checks the overall systemd state. The `systemctl --failed` command lists units currently in a failed state. Although both results indicated that Ubuntu was operational according to systemd, they did not establish that the system was free of networking or configuration problems.

Conclusion: No failed systemd units were detected, so I continued investigating the system's network configuration.



2.3 Investigating DNS Configuration
I examined Ubuntu's DNS configuration using the `cat` command displays the contents of a file in the terminal.

Command executed: *cat /etc/resolv.conf*
The output contained: nameserver 999.999.999.999

This was an invalid IPv4 address and matched the DNS modification included in Scenario 1 of the professor's breaker script.

Conclusion: I identified a damaged DNS configuration that could prevent Ubuntu from resolving domain names.

```
jrodriguez@linuxadminclass-ubuntu:~$ cat /etc/resolv.conf
nameserver 999.999.999.999
```

2.4 Investigating the Routing Table
Next, I examined the network routing table: *ip route*

The `ip route` command displays the system's IPv4 routing entries. The output showed that Ubuntu was using interface `enp0s3` with IP address `10.0.0.225`. However, the routing table did not contain a default route. Although a route to `10.0.0.1` was present, this was not equivalent to having a default gateway.

Conclusion: Ubuntu had no default route for sending traffic to destinations outside its directly connected network.

```
jrodriguez@linuxadminclass-ubuntu:~$ ip route
10.0.0.0/24 dev enp0s3 proto kernel scope link src 10.0.0.225 metric 1024
10.0.0.1 dev enp0s3 proto dhcp scope link src 10.0.0.225 metric 1024
75.75.75.75 via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.225 metric 1024
75.75.76.76 via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.225 metric 1024
```

2.5 Confirming the Network Failure
I tested connectivity to an external IP address using the following commands recorded in my history: *ping -c 4 1.1.1.1*

The `ping` command sends ICMP echo requests to a destination. The `-c` option specifies the number of requests.

My recorded four-packet test returned: Network is unreachable

Because the test used an IP address instead of a domain name, it did not depend on DNS resolution.

Conclusion: The failed connectivity test supported my earlier finding that Ubuntu's missing default route was preventing external communication.

2.6 Examining the Netplan Configuration Files

I investigated Ubuntu's Netplan configuration directory: *ls -l /etc/netplan/*

The `ls` command lists directory contents, and the `-l` option displays detailed file information, including permissions and ownership.

The output showed two configuration files:
“00-installer-config.yaml”
“99-broken.yaml”

I recognized `99-broken.yaml` as the filename used by Scenario 1 of the professor's script.

I examined it using: sudo cat /etc/netplan/99-broken.yaml

The file contained the following configuration:

```
network:
  version: 2
  renderer: networkd
  ethernets:
    eth0:
      dhcp4: no
      addresses: [192.168.1.500/24]
      gateway4: 192.168.1.1
      nameservers:
        addresses: [999.999.999.999]
```

I identified several problems, including an invalid IP address, an invalid DNS address, DHCP being disabled for `eth0`, and a gateway that did not match my existing network.

I then examined the original configuration: *sudo cat /etc/netplan/00-installer-config.yaml*

The original file configured `enp0s3` to use DHCP for IPv4 and IPv6. It also contained a MAC address used to match the network interface.

Conclusion: The presence and contents of `99-broken.yaml`, together with the invalid DNS configuration and missing default route, provided strong evidence that Scenario 1 — Networking had been selected.

```
jrodriguez@linuxadminclass-ubuntu:~$ sudo cat /etc/netplan/00-installer-config.yaml
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: true
      dhcp6: true
      match:
        macaddress: 08:00:27:fe:99:c5
      set-name: enp0s3
  version: 2
jrodriguez@linuxadminclass-ubuntu:~$ sudo mv /etc/netplan/99-broken.yaml /root/99-broken.yaml.bak
jrodriguez@linuxadminclass-ubuntu:~$ _
```

2.7 Removing the Damaged Netplan Configuration
After identifying the suspicious configuration, I moved it out of Netplan's configuration directory: *sudo mv /etc/netplan/99-broken.yaml /root/99-broken.yaml.bak*

The `mv` command moves or renames files. I moved the damaged configuration into the root user's directory and renamed it with a `.bak` extension so I could preserve it as evidence.

I verified the directory contents afterward: ***ls -l /etc/netplan/***

The output showed that only `00-installer-config.yaml` remained.

Conclusion: I successfully removed the damaged file from Netplan's configuration directory without permanently deleting the evidence.

```
jrodriguez@linuxadminclass-ubuntu:~$ ls -l /etc/netplan/
total 4
-rw------- 1 root root 209 Sep  9 23:41 00-installer-config.yaml
jrodriguez@linuxadminclass-ubuntu:~$ _
```

2.8 Encountering a Netplan Interface-Matching Error
I attempted to apply the original network configuration: *sudo netplan apply*

The `netplan apply` command applies the current Netplan configuration to the system.

However, the command returned: “Cannot find unique matching interface for enp0s3”

This indicated that Netplan could not match the configured interface using its existing settings.

I investigated the interface using: *ip link show enp0s3*

The `ip link show` command displays information about a network interface, including its state and hardware address.

The output showed that the actual interface MAC address was: 08:00:27:24:0a:bc. The original Netplan configuration contained a different MAC address:08:00:27:fe:99:c5 Because I was using a VirtualBox clone, I determined that the clone's MAC address did not match the address saved in the original configuration.

Conclusion: The MAC address mismatch explained why Netplan could not apply the original configuration successfully.

```
jrodriguez@linuxadminclass-ubuntu:~$ ip link show enp0s3
2: enp0s3: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc pfifo_fast state UP mode DEFAULT group default qlen 1000
    link/ether 08:00:27:24:0a:bc brd ff:ff:ff:ff:ff:ff
    altname enx080027240abc
```

2.9 Correcting the Network Interface Configuration
I opened the original Netplan configuration for editing:
sudo nano /etc/netplan/00-installer-config.yaml

The `nano` command opens a terminal-based text editor. I used `sudo` because the configuration file requires administrator privileges to modify. I changed the MAC address from 08:00:27:fe:99:c5 to08:00:27:24:0a:bc. I left the remaining configuration unchanged, including the IPv4 and IPv6 DHCP settings.

After saving the file, I applied the configuration again:*sudo netplan apply*

This time, the command returned to the terminal prompt without displaying an error.
Conclusion: Correcting the MAC address allowed Netplan to apply the original DHCP-based network configuration.

```
# This is the network config written by 'subiquity'
network:
  ethernets:
    enp0s3:
      dhcp4: true
      dhcp6: true
      match:
        macaddress: 08:00:27:24:0a:bc
      set-name: enp0s3
  version: 2
```

2.10 Verifying the Restored Routing Configuration
I checked the routing table again: *ip route*

This time, the output included:
default via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.61 metric 100

The default gateway had been restored to `10.0.0.1`, and Ubuntu had obtained IP address `10.0.0.61` through DHCP.

Conclusion: The corrected Netplan configuration successfully restored Ubuntu's default route.

```
jrodriguez@linuxadminclass-ubuntu:~$ ip route
default via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.61 metric 100
10.0.0.0/24 dev enp0s3 proto kernel scope link src 10.0.0.61 metric 100
10.0.0.1 dev enp0s3 proto dhcp scope link src 10.0.0.61 metric 100
75.75.75.75 via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.61 metric 100
75.75.76.76 via 10.0.0.1 dev enp0s3 proto dhcp src 10.0.0.61 metric 100
```

2.11 Verifying External Network Connectivity
I repeated the external connectivity test: *ping -c 4 1.1.1.1*

This time, all four packets were received successfully.

The results showed:

4 packets transmitted, 4 received, 0% packet loss

The average round-trip time was approximately 21.035 milliseconds.

Conclusion: Ubuntu could now communicate with an external IP address, confirming that external network connectivity had been restored.

```
jrodriguez@linuxadminclass-ubuntu:~$ ping -c 4 1.1.1.1
PING 1.1.1.1 (1.1.1.1) 56(84) bytes of data.
64 bytes from 1.1.1.1: icmp_seq=1 ttl=57 time=17.9 ms
64 bytes from 1.1.1.1: icmp_seq=2 ttl=57 time=38.2 ms
64 bytes from 1.1.1.1: icmp_seq=3 ttl=57 time=9.61 ms
64 bytes from 1.1.1.1: icmp_seq=4 ttl=57 time=18.5 ms

--- 1.1.1.1 ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 3004ms
rtt min/avg/max/mdev = 9.613/21.035/38.176/10.497 ms
```

2.12 Verifying DNS Configuration and Name Resolution
After restoring network connectivity, I checked the DNS configuration again: *cat /etc/resolv.conf*

The file no longer contained the invalid DNS server. Instead, it displayed:
nameserver 127.0.0.53
options edns0 trust-ad
search

The `127.0.0.53` address is the local DNS stub resolver used by `systemd-resolved`.

To verify that domain-name resolution worked, I executed: *getent hosts google.com*

The `getent` command retrieves entries from system databases. With the `hosts` database, it can resolve a hostname to an IP address using the system's configured name-resolution mechanisms.

The command returned: 2607:f8b0:4006:802::200e google.com

Conclusion: Ubuntu successfully resolved `google.com` to an IPv6 address, confirming that name resolution was functioning.

```
jrodriguez@linuxadminclass-ubuntu:~$ getent hosts google.com
2607:f8b0:4006:802::200e google.com
```

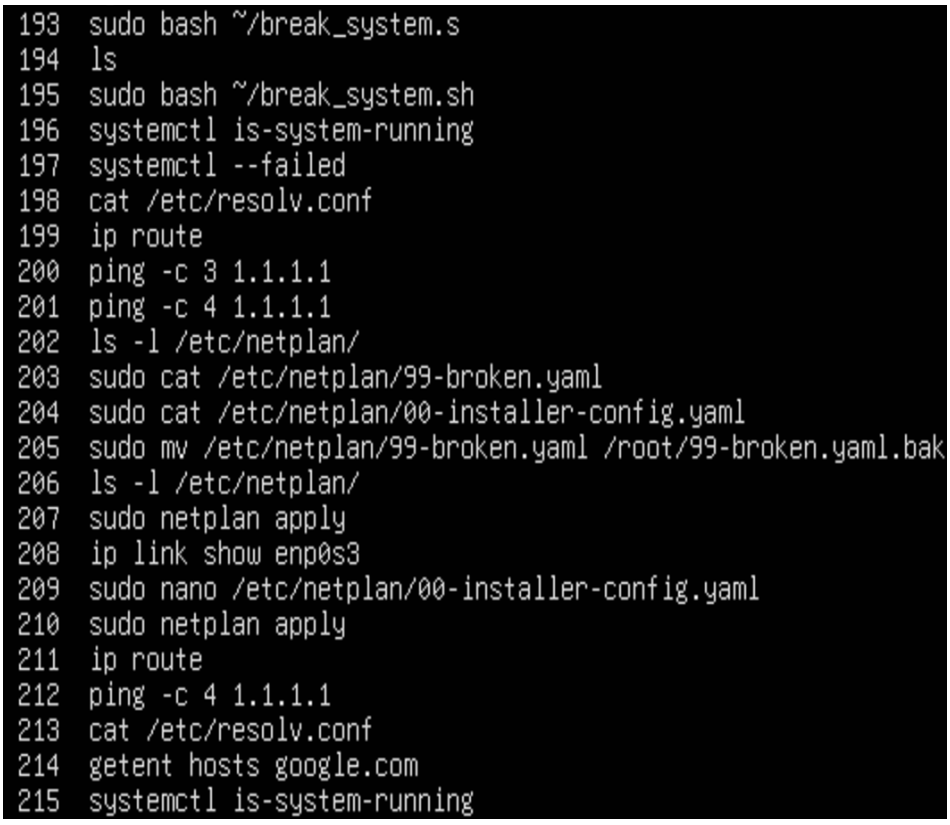
2.13 Final System Health Verification
I performed one final system health check: *systemctl is-system-running*
The command returned: running

This confirmed that systemd considered Ubuntu operational after the network repairs.

Conclusion: The network repair restored the default route, external IP connectivity, and DNS resolution while leaving the system in a running state.

2.14 Reviewing the Troubleshooting Command History
I captured my troubleshooting history using:
history | tail -n 60

The `history` command displays commands previously executed in Bash. The pipe (`|`) sends that output to `tail`, and `-n 60` requests the most recent 60 entries. The provided screenshot showed history entries 193–215, including the script execution, initial diagnosis, unsuccessful network tests, Netplan repair, and final verification. I preserved the command history to demonstrate the troubleshooting process, including commands that initially failed.



2.15 Final Ubuntu Investigation Summary

During my Ubuntu investigation, I identified Scenario 1 — Networking as the failure introduced by the professor's `break_system.sh` script.

The system initially reported that it was running and had no failed systemd units. However, further investigation revealed an invalid DNS server, a missing default route, and an inability to communicate with an external IP address.

I discovered that the script had created a damaged Netplan configuration file named `99-broken.yaml`. I preserved this file outside the Netplan directory and attempted to restore the original network configuration.

During the repair, I encountered an additional issue caused by the VirtualBox clone having a different MAC address from the one recorded in the original Netplan configuration. After correcting the MAC address and applying Netplan again, Ubuntu successfully obtained an IP address and restored its default gateway.

I verified the repair using the routing table, a successful external ping test with 0% packet loss, a successful DNS lookup, and a final system health check.

These results demonstrated that the identified networking problems had been repaired successfully.

Sources: System Breaker Script — “break_system.sh”
https://netplan.readthedocs.io/

3. Ubuntu — Script Transfer and Testing

3.1 Transferring the Script to Ubuntu
After testing the script on CentOS, I transferred the same system_diag.sh file to my Ubuntu virtual machine.
The script was placed in:
/home/jrodriguez/Security_Lab/scripts/
I did not need to modify the diagnostic commands for Ubuntu because the script used commands available on both Linux distributions.
This allowed me to test whether the same script could collect system information consistently across both servers.

3.2 Executing the Script on Ubuntu
I navigated to the scripts directory and executed the script using:
bash system_diag.sh
The terminal displayed:
Diagnostics completed successfully.
Report saved to: /home/jrodriguez/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
This confirmed that the script reached its final completion message on Ubuntu.
The generated report was saved in the same directory structure used on CentOS, but under the Ubuntu user's home directory.

3.3 Verifying the Seven Report Sections
I verified that the Ubuntu report contained all seven sections using:

grep -n '===== ' ~/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
The command displayed the following results:

Section	Starting line
System Information	1
CPU and Memory Information	22
Disk Space Information	74
Network Diagnostics	102
Systemd Services	168
System Logs and Errors	198
Diagnostics Report Complete	244

This confirmed that all seven diagnostic sections were included in the Ubuntu report.

3.4 Checking for Execution Errors

After verifying the report sections, I searched the report for common execution errors using:
grep -niE 'permission denied|command not found|no such file|failed to execute' ~/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
The command returned directly to the terminal prompt without displaying any matches.
This indicated that the report did not contain the error messages included in my search.
Although this does not guarantee that every command executed without any possible issue, it provides additional evidence that the script ran as expected.

3.5 Reviewing the Network Diagnostics

I reviewed the networking and systemd portions of the report using:
sed -n '102,197p' ~/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
The network diagnostics showed that Ubuntu successfully communicated with the external IP address 1.1.1.1.
The ping test returned:
4 packets transmitted, 4 received, 0% packet loss
The average response time was approximately 7.585 milliseconds.

The DNS resolution test also successfully resolved google.com to an IPv6 address. These results confirmed that Ubuntu had working external IP connectivity and hostname resolution during the test.
This was especially useful because the Ubuntu server had previously experienced networking problems during the troubleshooting portion of the lab.

3.6 Reviewing Systemd Services
The systemd section of the Ubuntu report displayed:
Overall system status:
running

Failed services:
0 loaded units listed.
The report also listed 22 running services.
These included:

- ssh.service
- systemd-networkd.service
- systemd-resolved.service
- chrony.service

This confirmed that the system was reporting a running state and that no failed systemd units were listed at the time of the test.
The service information also demonstrated that the script could identify important running services.

3.7 Reviewing System Logs and Errors
I reviewed the system logs and final report summary using:
sed -n '198,247p' ~/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
The journal entries included several warnings and errors.
These included earlier Chrony time synchronization failures, a systemd network wait-online timeout, virtual graphics compatibility warnings, an invalid hostname warning, and kernel workqueue warnings.
The Chrony messages showed that the server had previously experienced difficulty communicating with time synchronization servers.
The network wait-online message indicated that the system had experienced a network startup timeout.
The graphics warnings were related to the virtualized environment.
The invalid hostname warning indicated that a hostname entry in /etc/hosts was not accepted by the resolver.

The kernel workqueue warnings indicated that some kernel tasks were taking longer than expected.
These messages were useful because they demonstrated that the script could capture potential problems for later investigation.
However, they did not prove that all of these problems were still active when the diagnostic report was generated.

3.8 Verifying the Saved Diagnostic Report

After reviewing the diagnostic output, I confirmed that the report file existed using:
ls -lh ~/Security_Lab/reports/system_diagnostics_2026-11-06_16-50-51.txt
The terminal showed that the report was approximately 16 KB in size.
The file belonged to the jrodriguez user and had the permissions:
-rw-rw-r--
This confirmed that the diagnostic report had been saved successfully and was available for later review.

3.9 Ubuntu Conclusion

I successfully executed the same Bash diagnostic script on Ubuntu without modifying its diagnostic commands.
The script generated a timestamped report containing all seven expected sections.
The report confirmed working network connectivity, successful DNS resolution, and no currently failed systemd units.
It also collected journal entries that could be useful for future troubleshooting.
This demonstrated that the script could perform the same diagnostic tasks on Ubuntu as it did on CentOS Stream 10.

3.10 Overall Diagnostic Script Conclusion

For this stage of the lab, I created and tested an automated Bash diagnostic script on both CentOS Stream 10 and Ubuntu.
The script collected information about the operating system, CPU, memory, disk space, networking, systemd services, and system logs.
Both servers successfully generated timestamped diagnostic reports containing all seven expected sections.
Testing the script on two Linux distributions demonstrated that the same diagnostic commands could be used without modifying the script for each operating system.

The reports also showed that automated diagnostics can provide useful troubleshooting evidence by collecting system information and recording warnings or errors in one location.
The next stage of the lab will involve scheduling the script with cron so that diagnostic reports can be generated automatically at regular intervals.

Sources: https://www.aholdengouveia.name/LinuxAdmin/serversecurity.html
https://web.archive.org/web/20220727074657/https://www.geeksforgeeks.org/ping-command-in-linux-with-examples/
https://web.archive.org/web/20220510004952/https://www.networkworld.com/article/3428361/how-to-manage-logs-in-linux.html
https://www.aholdengouveia.name/LinuxAdmin/networking.html
https://www.gnu.org/software/bash/manual/bash.html
https://ubuntu.com/server/docs/explanation/networking/configuring-networks/
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html
https://www.aholdengouveia.name/LinuxAdmin/networking.html
https://www.gnu.org/software/coreutils/manual/coreutils.html

```bash
#!/bin/bash

# Compatible with CentOS and Ubuntu
# Purpose: Collect health information for troubleshooting

REPORT_DIR="$HOME/Security_Lab/reports"
TIMESTAMP=$(date +"%Y-%m-%d_%H-%M-%S")
REPORT_FILE="$REPORT_DIR/system_diagnostics_$TIMESTAMP.txt"

mkdir -p "$REPORT_DIR"

# Section 1: Basic System Informaton

{
        echo "======= SYSTEM INFORMATION ======="
        echo "Report generated: $(date)"
        echo "Hostname: $(hostname)"
        echo "Operating System: "
        cat /etc/os-release
        echo "Kernel version: $(uname -r)"
        echo "System uptime: "
        uptime
} >> "$REPORT_FILE" 2>&1

# Section 2: CPU and Memory Information

{
        echo " "
        echo "======= CPU AND MEMORY INFORMATION ======="
        echo "CPU details: "
        lscpu
        echo "Memory usage: "
        free -h
        echo "System load: "
        uptime
} >> "$REPORT_FILE" 2>&1
```

```
# Section 3: Disk Space Information

{
        echo " "
        echo "======= DISK SPACE INFORMATION ======="
        echo "Filesystem disk usage: "
        df -h
        echo "Disk and partition information: "
        lsblk
} >> "$REPORT_FILE" 2>&1

# Section 4: Network Diagnostics

{
        echo " "
        echo "======= NETWORK DIAGNOSTICS ======="
        ip addr
        echo "Routing table: "
        ip route
        echo "DNS configuration: "
        cat /etc/resolv.conf
        echo "External connectivity test: "
        ping -c 4 -W 2 1.1.1.1
        echo "DNS resolution test: "
        getent hosts google.com
} >> "$REPORT_FILE" 2>&1
```

```
# Section 5: Systemd Services

{
        echo " "
        echo "======= SYSTEMD SERVICES ======="
        echo "Overall system status: "
        systemctl is-system-running
        echo "Failed services: "
        systemctl --failed --no-pager
        echo "Running services: "
        systemctl list-units --type=service --state=running --no-pager
} >> "$REPORT_FILE" 2>&1

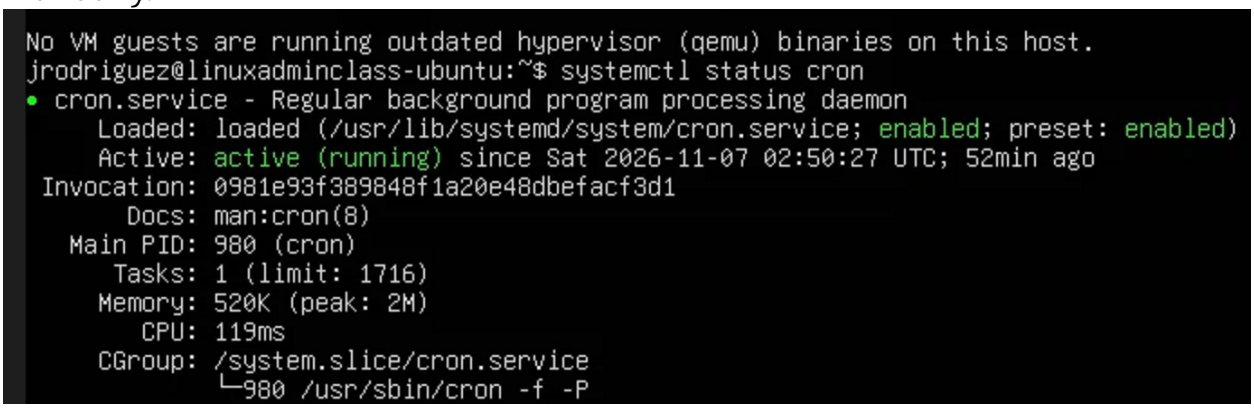
# Section 6: System Logs and Errors

{
        echo " "
        echo "======= SYSTEM LOGS AND ERRORS ======="
        echo "Recent system errors: "
        journalctl -p err -n 20 --no-pager
        echo "Recent boot warnings and errors: "
        journalctl -b -p warning -n 20 --no-pager
} >> "$REPORT_FILE" 2>&1

# Section 7: Final Report Summary
{
        echo " "
        echo "======= DIAGNOSTICS REPORT COMPLETE ======="
        echo "Report completed: $(date)"
        echo "Hostname: $(hostname)"
        echo "Review the sections above for potential issues."
} >> "$REPORT_FILE" 2>&1

echo "Diagnostics completed successfully."
echo "Report saved to: $REPORT_FILE"
```

4. Automated Diagnostic Scheduling with Cron
Ubuntu — Cron Configuration and Testing
4.1.1 Verifying the Cron Service
After successfully transferring and testing my automated diagnostic script on Ubuntu, I configured the system to execute the script automatically using cron. Cron is a Linux scheduling utility that allows administrators to run commands and scripts at scheduled intervals without requiring manual execution.
Before creating a scheduled task, I checked whether the cron service was running by executing:
systemctl status cron
The terminal displayed:
cron.service - Regular background program processing daemon
Loaded: loaded (/usr/lib/systemd/system/cron.service; enabled; preset: enabled)
Active: active (running)
This confirmed that the cron service was installed, active, and enabled to start automatically during system startup.
Since the service was already running, I did not need to install, start, or enable it manually.



4.1.2 Checking Existing Cron Jobs
Before creating a scheduled task, I checked whether my Ubuntu user account already had any cron jobs configured.
I executed:
crontab -l
The terminal returned:
no crontab for jrodriguez
This indicated that my user account did not have an existing personal crontab.

Checking for existing jobs helped ensure that I would not accidentally interfere with previously configured tasks.

```
jrodriguez@linuxadminclass-ubuntu:~$ crontab -l
no crontab for jrodriguez
```

4.1.3 Creating a Temporary Cron Schedule

To schedule the diagnostic script, I opened my personal crontab using:
crontab -e
I added the following entry:
** * * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh*
The five asterisks instruct cron to execute the command every minute.
I used the full path to the Bash interpreter and the diagnostic script to ensure that cron could locate both files.
I temporarily scheduled the script to run every minute so I could verify that automatic execution worked without waiting for the final daily schedule.
After entering the command, I saved and exited the editor.
The terminal displayed:
crontab: installing new crontab
This confirmed that the new scheduled task was installed.

```
* * * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh
crontab: installing new crontab
jrodriguez@linuxadminclass-ubuntu:~$ _
```

4.1.4 Verifying the Temporary Schedule

After saving the cron entry, I verified that it was installed correctly using:
crontab -l
The terminal displayed:
** * * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh*
This confirmed that the diagnostic script was scheduled to execute every minute.

```
jrodriguez@linuxadminclass-ubuntu:~$ crontab -l
* * * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh
```

4.1.5 Testing Automatic Report Generation

After configuring the temporary schedule, I checked whether cron was automatically executing my diagnostic script.
I ran:
ls -lht ~/Security_Lab/reports/
The terminal displayed the following diagnostic reports:

Report filename	File size
system_diagnostics_2026-11-07_03-50-01.txt	16 KB
system_diagnostics_2026-11-07_03-49-01.txt	16 KB
system_diagnostics_2026-11-07_03-48-01.txt	16 KB
system_diagnostics_2026-11-07_03-47-01.txt	16 KB
system_diagnostics_2026-11-06_16-50-51.txt	16 KB

The timestamps showed that a new diagnostic report was created every minute, matching the temporary cron schedule.
The older report was from my previous manual execution of the diagnostic script.
These results provided evidence that Ubuntu's cron service was successfully executing the diagnostic script automatically without manual intervention.

```
jrodriguez@linuxadminclass-ubuntu:~$ ls -lht ~/Security_Lab/reports/
total 80K
-rw-rw-r-- 1 jrodriguez jrodriguez 16K Nov  7 03:50 system_diagnostics_2026-11-07_03-50-01.txt
-rw-rw-r-- 1 jrodriguez jrodriguez 16K Nov  7 03:49 system_diagnostics_2026-11-07_03-49-01.txt
-rw-rw-r-- 1 jrodriguez jrodriguez 16K Nov  7 03:48 system_diagnostics_2026-11-07_03-48-01.txt
-rw-rw-r-- 1 jrodriguez jrodriguez 16K Nov  7 03:47 system_diagnostics_2026-11-07_03-47-01.txt
-rw-rw-r-- 1 jrodriguez jrodriguez 16K Nov  6 16:50 system_diagnostics_2026-11-06_16-50-51.txt
```

4.1.6 Changing the Schedule to Run Daily

After confirming that the temporary schedule worked, I changed the cron job to execute once daily instead of every minute.
I reopened the crontab using:
crontab -e
I replaced the temporary entry with:
*0 2 * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh*
The five scheduling fields represent the minute, hour, day of the month, month, and day of the week.
The 0 specifies minute zero, while 2 specifies the second hour of the day. The remaining asterisks indicate every day of the month, every month, and every day of the week.
This configuration schedules the script to execute daily at 2:00 AM according to Ubuntu's system time.
I selected a daily schedule because it provides regular diagnostic reports without generating unnecessary files every minute.

After saving the updated entry, I verified it using:
crontab -l
The terminal displayed:
*0 2 * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh*
This confirmed that the daily schedule was installed correctly.

```
jrodriguez@linuxadminclass-ubuntu:~$ crontab -l
0 2 * * * /bin/bash /home/jrodriguez/Security_Lab/scripts/system_diag.sh
```

4.1.7 Ubuntu Cron Configuration Conclusion
I successfully configured and tested automated diagnostic scheduling on Ubuntu. I first verified that the cron service was active and enabled. I then confirmed that my user account did not have any existing personal cron jobs.
Next, I created a temporary schedule to execute my diagnostic script every minute. By examining the reports directory, I confirmed that four new diagnostic reports were generated at one-minute intervals.
This configuration allows Ubuntu to automatically collect system diagnostic information and save timestamped reports for future troubleshooting, provided the server and cron service are running at the scheduled time.

Sources: https://www.process.st/checklist/server-security-checklist/
https://man7.org/linux/man-pages/man5/crontab.5.html
https://man7.org/linux/man-pages/man1/crontab.1.html
https://www.freedesktop.org/software/systemd/man/latest/systemctl.html
https://manpages.ubuntu.com/manpages/noble/en/man5/crontab.5.html

Ubuntu 26.04.1 LTS — Emergency/Recovery-Mode Guide

Executive Summary

During this lab, I successfully accessed Ubuntu's recovery environment using the GRUB bootloader on my VirtualBox clone. I examined the available recovery options, accessed the root maintenance shell, checked the root filesystem's mount status, and restarted Ubuntu successfully. This procedure demonstrates how a Linux administrator can access recovery tools when a server experiences startup problems.

Emergency/Recovery-Mode Procedure

1. Verify the normal boot configuration.

I checked Ubuntu's default systemd target using:

systemctl get-default

The result was graphical.target, confirming that Ubuntu normally starts in its graphical operating mode.

2. Access the GRUB recovery menu.

I restarted Ubuntu using:

sudo reboot

During startup, I accessed GRUB version 2.14 and selected **Advanced options for Ubuntu**.
I then selected:
Ubuntu, with Linux 7.0.0-42-generic (recovery mode)
Ubuntu displayed its Recovery Menu, which included options for freeing disk space (clean), repairing packages (dpkg), checking filesystems (fsck), updating GRUB (grub), enabling networking (network), and opening a root shell (root).

3. Access and inspect the root maintenance shell.

I selected root — Drop to root shell prompt and successfully accessed:
root@linuxadminclass-ubuntu:~#
I checked the root filesystem using:
findmnt -no TARGET,OPTIONS /
The output was:
/ rw,relatime
This confirmed that the root filesystem was mounted read-write at the time of inspection, even though the initial recovery menu had displayed a read-only filesystem state.
I did not modify any system files because no repairs were necessary.

4. Return Ubuntu to normal operation.

After verifying the recovery environment, I restarted Ubuntu using:
reboot
Ubuntu successfully returned to its normal login prompt, displaying Ubuntu 26.04.1 LTS and the hostname linuxadminclass-ubuntu.

Figure 1:

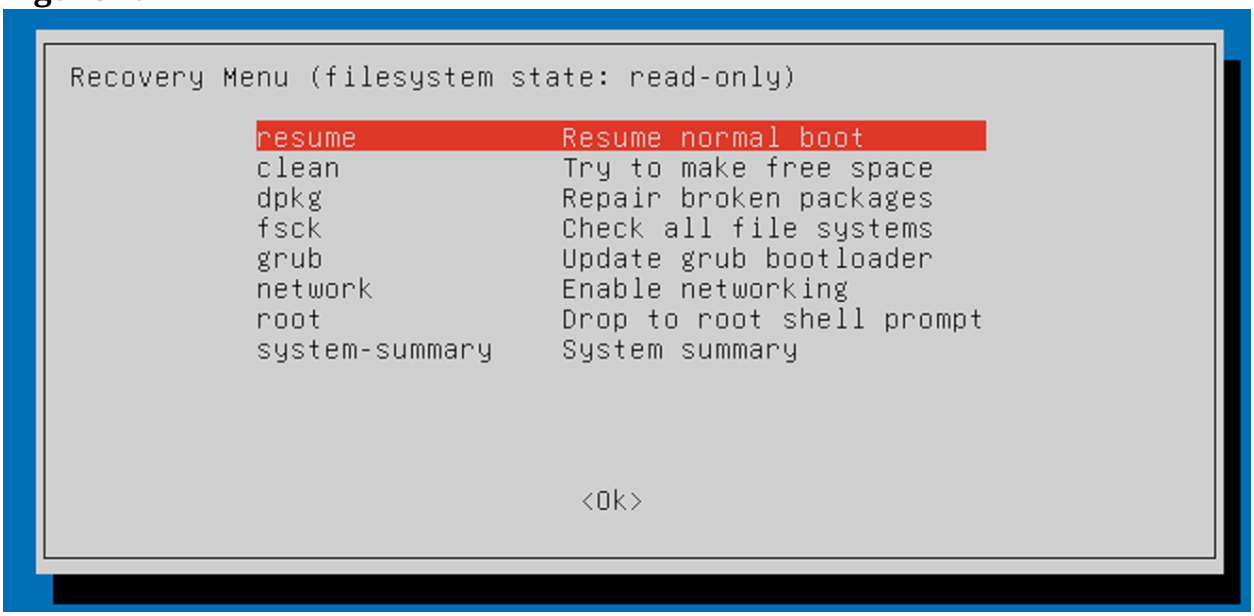


Figure 2:

```
root@linuxadminclass-ubuntu:~# findmnt -no TARGET,OPTIONS /
/ rw,relatime
```

Sources: https://wiki.ubuntu.com/Recovery_mode
https://man7.org/linux/man-pages/man8/findmnt.8.html